BSDA2001P: Introduction to DL and GenAI Project

1. Executive Summary

This report is for Smart MCQ challenge on kaggle to predict multiple choice questions. Each question contains a prompt and five labels (e.g. A, B, C, D, E).The Training Dataset contains 2000 questions and 8 Columns while the test dataset contains 500 questions and 7 columns. The evaluations are based on MAP@3. Which identify 3 likely choices.

Initial analysis showed that the data doesn't have any missing values in any column.The distribution of the correct answer in training data was A - 369, B - 490, C - 459, D - 358, E - 324. In preprocessing duplicates questions were removed using the prompt column. The question and all 5 options were combined into a single text. After removing duplicates the training datasets are divided into 80:20 using train test split which is 1406 training samples and 352 validation samples with fixed random state is 42.

Three Distinct models were used (Model from scratch, Pretrained model, Additional model). The first model was a multi layer perceptron(MLP) neural network built from scratch, using TF-IDF features. The second model was a fine tuned DistilBert transformer model. The third was an XGBoost classifier trained on the same TF-IDF. The MLP achieved a validation accuracy and weightened F1-score of 1.00. DistilBert achieved 99.43% validation accuracy and 0.9943weighted F1 score after 3 epochs. XGBoost also achieved 99.43% validation accuracy and 0.99432 on weighted F1-score.

For Final submission Probability based ensembling was used. DistilBert contributed 40%, XGBoost contributed 40% and MLP contributed 20%. The three likely possible answers were given from these models. The submission files contain 500 test questions.

Both text-based machine learning models and transformer-based deep learning can perform strongly on this data. Different models can produce useful probability estimation on the same text data, which makes ensemble-based ranking a neutral strategy for a top 3 prediction problem.

BSDA2001P: Introduction to DL and GenAI Project

2. Introduction

2.1. Problem statement

Multiple-choice question solving requires a model to understand the question, compare it with all options and identify the best 3 options. The challenge transforms this problem into a supervised machine learning task.

The training data have questions with 5 options and a labelled correct answer. The test data has the same structure but does not contain the answer column. The training data has the columns id, prompt, A, B, C, D, E and answer, whereas the test data only contains id, prompt, and the five options.

This competition uses MAP@3 as the primary evaluation metric. Therefore, producing only single-class predictions is not sufficient for the final competition objective. So we need to rank its predictions for the correct answer that appears within the first three positions.

2.2. Project Objective

The main objective of this project is to build an MCQ-solving pipeline that is able to produce the top 3 ranked answers. This project also provides practical experience with several different model approaches.

Objectives:

1. Understand the structure of competition data and explore also
2. Clean duplicate questions and prepare for text data
3. Transform the MCQ info for model
4. Experiment with TF-IDF-based machine learning
5. Build custom neural network with pytorch
6. Fine tuned pretrained model from hugging face
7. Experiment with XGBoost as an additional classification model
8. Track using weights and biases
9. Combine model probabilities to create top 3 predictions
10. Deployment on Streamlit

2.3. Report Structure

The report is all about preprocessing and then explains the tokenizer strategy used for the transformer model. The modeling section covers the custom MLP, DistilBert, and XGBoost, and then performance and future improvement are also discussed.

3. Dataset and Preprocessing

3.1. Dataset Description

In this competition we have 2 CSV files train.csv and test.csv

Training Data

1. 2000 rows
2. 8 columns

Column	Description
id	Unique question number
prompt	Question text
Five column for option (A, B, C, D, E)	options
answer	Correct answer label

Test Data

1. 500 rows
2. 7 columns

In test data we don't have an answer column because we need to predict it.

3.2. Exploratory Data Analysis

The first step was inspecting the shape, columns, data types of both data. The id was integer-based while options and question prompts were stored as string/object.

Answer	Count

A	369
B	490
C	459
D	358
E	324

The data is not balanced because option B occurs most frequent and option E least frequent. So a Stratified split was used to preserve the class distribution between training and validation data.

3.3. Data Preprocessing

There are no missing values in the dataset. So no imputation was required.

Duplicate prompts are removed using drop_duplicates, so it reduces the repeated questions. After this we combine questions and five possible answers into text data.

Question: [prompt] Options: A) [A] B) [B] C) [C] D) [D] E) [E]

We apply this for train and test data. After we divide our data into train and test with validation size 20% random state 42.

1. Training samples: 1406
2. Validation samples: 352

The answer labels were converted into numerical class id using LabelEncoder.

4. Tokenization Strategy

4.1. Primary Tokenizer Selection

The primary tokenizer was DistilBert tokenizer associated with DistilBert-base-uncased. This tokenizer is loaded from the Hugging face transformer library and it is used for converting the combined MCQ text into token ids so it can be processed by the transformer model. The preprocessing pipeline used a maximum sequence length of 512 Tokens. Sequences longer than

the limit were truncated, while short sequences were padded to the maximum length.

Justification:

The MCQ inputs contain natural language questions as well as long answer choices. The transformer tokenizer is useful because it preserves the sequence structure required by transformer architecture and allows pretrained models to process the question and answer choices together.

Using pretrained tokenizers also avoids building vocabulary from scratch.This is particularly useful for small training data.

4.2. Implementation Details

The tokenizer function used padding, truncation, max length. The hugging face interface was used to tokenize the training and validation data in batches. During test inference, batches of 16 questions were tokenized at a time, with the same maximum sequence length of 512.

4.3. Experimental Comparison

The custom MLP and XGBoost models did not use transformer tokenization. They used a TF-IDF vectorizer with a maximum of 5,000 features and English stop-word removal.

This created two different text-processing approaches in the project:

1. TF-IDF → MLP/XGBoost
2. DistilBERT tokenizer → Transformer

5. Modeling & Experimentation

5.1. Model 1: Fine tuned transformer - DistilBert

The transformer model used in this project was DistilBERT, initialized from the pretrained distilbert-base-uncased.

The model was adapted for five-class sequence classification:- Class A, Class B, Class C, Class D, Class E

The classification head was therefore configured with five output labels.

DistilBERT uses the transformer encoder architecture. Its self-attention mechanism allows the model to consider relationships between different parts of the input sequence. This is useful for MCQs because the meaning of an answer choice depends on the question and, in many cases, on specific words elsewhere in the combined input.

Fine-Tuning Strategy

The model was fine-tuned using the Hugging Face Trainer.

Parameter	Value
Model	distilbert-base-uncased
Number of classes	5
Epochs	3
Batch size	16
Maximum sequence length	512
Evaluation	Every epoch
Model Selection	Best validation accuracy
Experimental tracking	Weights & Biases

No layers were explicitly frozen. The pretrained model was fine-tuned on the MCQ dataset.

Results

The model showed a large improvement during the three training epochs.

Epoch	Validation Loss	Accuracy	F1
1	1.903575	0.823864	0.823938
2	0.386182	0.994318	0.994297
3	0.133388	0.994318	0.994322

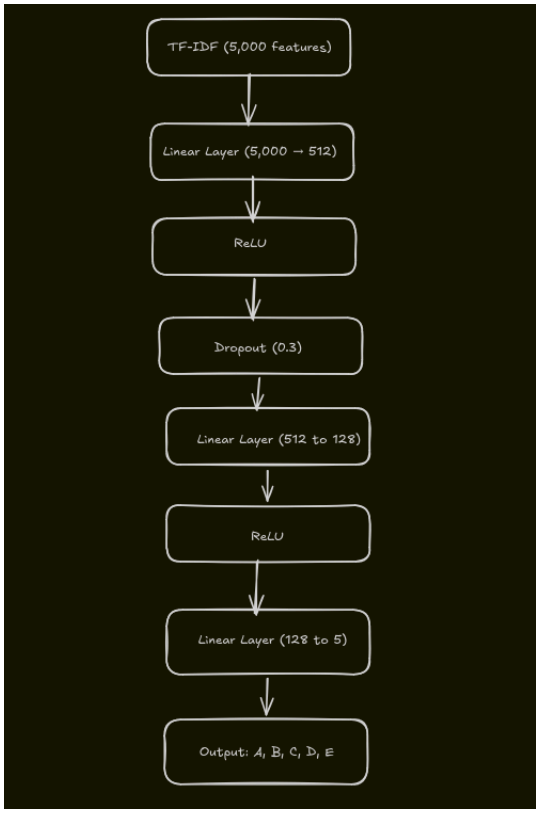
The final validation accuracy was 99.43%, with a weighted F1-score of approximately 0.9943. The W&B run also recorded the evaluation loss, accuracy, F1, runtime, training epoch, and global training steps.

Quantization

No explicit model quantization was applied in the notebook.

5.2. Model 2: Custom deep learning model - MLP

Architecture



The second major approach was a custom neural network built from scratch using PyTorch. The input to the network was the TF-IDF representation containing up to 5,000 features.

Training Strategy

Optimizer: AdamW

Learning rate: 0.001

Loss: Cross Entropy Loss

Batch size: 32

Epochs: 15

Dropout: 0.3

Result

Validation accuracy: 1.000

Weighted accuracy: 1.000

The result shows that the TF-IDF representation contained strong discriminative information for this dataset. However, because the validation accuracy is exactly 100%, the result should be interpreted carefully. A high validation score on a relatively small dataset does not automatically guarantee the same performance on unseen competition questions.

5.3. Model 3: Additional Model - XGBoost

XGBoost was trained using the TF-IDF representation.

Parameter	Value
Estimators	500
Learning rate	0.05
Maximum depth	6
Random state	42

The model achieved:

Accuracy: 0.99432

Weighted F1: 0.99432

This result was very close to the DistilBERT result.

6. Performance & Comparative Analysis

6.1. Evaluation metrics

Accuracy

Accuracy measures the proportion of validation samples for which the model predicts the correct answer class.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Weighted F1-score

F1-score combines precision and recall. The weighted version calculates the F1-score for each class and weights the result according to the number of samples in that class.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

MAP@3

The primary competition metric is Mean Average Precision at 3.

The metric evaluates the ranking of the first three predictions. A correct answer ranked first receives the strongest contribution, while a correct answer appearing second or third contributes less.

6.2. Training Performance

Model	Input Representation	Accuracy	Weighted F1
MLP	TF-IDF	1.00	1.00
DistilBert	Transformer tokens	0.9943	0.9943
XGBoost	TF-IDF	0.9943	0.9943

The MLP produced the highest validation accuracy among the implemented models. However, the difference between the MLP and the other two models was small in absolute terms.

6.3. Comparative Report

The three implemented approaches have different characteristics.

The MLP was relatively simple and computationally lightweight compared with a transformer. Its TF-IDF input provided a high-dimensional representation of the text, and the neural network learned a nonlinear mapping from these features to the five answer classes.

DistilBERT required substantially more computation but provided a pretrained language representation. Its validation performance was very strong and it improved rapidly during fine-tuning.

XGBoost provided another perspective on the same TF-IDF features. Its performance was almost identical to DistilBERT on the validation set, which made it useful as part of the final ensemble.

Rather than assuming that the highest validation accuracy alone should determine the final model, the project combined the three probability distributions. This was particularly useful because the competition evaluates ranked top-3 predictions.

6.4. Kaggle Performance

Leaderboard Rank: 132

Total Participants: 1531

The validation accuracy and F1 values reported above should not be substituted for the Kaggle MAP@3 score.

7. Conclusion & Future Work

7.1. Key learnings

7.1.1. The project provided practical experience in several areas:

7.1.2. Handling and inspecting a real competition dataset.

7.1.3. Detecting and removing duplicate textual samples.

7.1.4. Creating a unified representation for question-answer pairs.

7.1.5. Using TF-IDF for text classification.

7.1.6. Building a neural network from scratch using PyTorch.

7.1.7. Fine-tuning a pretrained transformer with Hugging Face.

7.1.8. Using XGBoost for text classification from TF-IDF features.

7.1.9. Managing GPU-based model training.

7.1.10. Using FP16 training for transformer fine-tuning.

7.1.11. Tracking experiments using Weights & Biases.

7.1.12. Combining multiple models through probability-level ensembles.

7.1.13. Generating ranked predictions for a top-k evaluation metric.

7.1.14. Saving model and preprocessing components for later inference.

7.2. Challenge faced

The data is very small (only 2000 rows), so I have encountered issues such as data leakage. For Deep learning transformers, we need more data to train our model effectively.

Another issue was the presence of many repeated prompts in the original data, so we removed all duplicates before the train-validation split to reduce redundant information.

The transformer model also produced checkpoint loading messages indicating newly initialized classification-layer parameters. These parameters were expected because the pretrained language model was being adapted to a new five-class classification task.

A further practical challenge was converting the individual model outputs into the required top-3 competition format. This was handled by combining the probability distributions and ranking the five answer classes.

7.3. Areas of improvement

7.3.1. Improve Ensemble Weight Selection

The current ensemble uses fixed weights:

DistilBERT: 40%

XGBoost: 40%

MLP: 20%

These weights were not selected through an extensive optimization procedure. A future experiment could search for better combinations using validation MAP@3.

7.3.2. Improve Validation Strategy

The current validation uses one stratified 80:20 split. Multiple validation splits or cross-validation could provide a more reliable estimate of generalization.

8. Reference

8.1. Weight and Bias Documentation

8.2. Pytorch Documentation